# Large-Scale k-mer Counting on Genomic Data with CUDA

## Exact 31-mer counting on a human chromosome with a GPU hash table

Suphachock Kamlek

01418343 Parallel Computing with CUDA
Computer Science

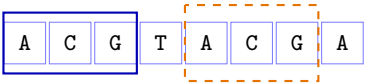
November 4, 2026

# Outline

1. The Problem
2. Why Not a Histogram?
3. Datasets
4. Sequential Solution
5. Parallel Solution
6. Benchmark
7. Optimizations
8. Future Work
9. Q & A

# What is a k-mer?

- A **k-mer** is a substring of length $k$ of a DNA sequence over $\{A, C, G, T\}$.
- **k-mer counting**: for every distinct k-mer, how many times does it occur?




$k = 3$: `ACG` $\times 2$, `CGT`, `GTA`,
`TAC`, `CGA` $\times 1$ $\quad (n - k + 1 = 6$ k-mers$)$

**Input / output**

- In: a genome in FASTA format and $k$
- Out: a table of (k-mer, count) pairs

**Used for**

- genome assembly (de Bruijn graphs)
- sequencing error correction
- genome size estimation, metagenomics
- alignment-free sequence comparison

# What we solve

## Goal

Count **all 31-mers exactly** in the human chromosome 1 (230 million k-mer positions) and make it fast with CUDA.

- $k = 31$: long enough to be (almost) unique in a genome, and fits in one 64-bit word ($2 \times 31 = 62$ bits).
- Exact counts, no probabilistic sketches.
- Must handle unknown bases (`N`): a k-mer may not cross an `N` run.
- The result must match a sequential reference **k-mer by k-mer**.

## Core difficulty

Hundreds of millions of concurrent "insert or increment" operations on a sparse key space.

# Counting with a histogram (direct addressing)

A histogram needs one bin per possible key:

$$\text{bins} = 4^k, \qquad \text{memory} = 4^k \times 4 \text{ bytes}$$

| $k$ | bins | memory |
|---|---|---|
| 12 | $1.7 \times 10^7$ | 67 MB |
| 15 | $1.1 \times 10^9$ | 4.3 GB |
| 16 | $4.3 \times 10^9$ | 17 GB |
| 31 | $4.6 \times 10^{18}$ | 18 EB |

- Fits a 16 GB GPU only up to $k \approx 15$.
- chr1 has $2.08 \times 10^8$ distinct 31-mers out of $4.6 \times 10^{18}$ possible: only $4.5 \times 10^{-11}$ of the bins would ever be used.
- The key space is **huge and sparse** $\Rightarrow$ we need a **hash table**: memory proportional to the k-mers that actually occur.

# Additional constraints

1. **Key + value is wider than one atomic**: a slot is `KeyValue{uint64_t key; uint64_t val;}`, key 62 bits (stored in 64) + count 64 bits = 128 bits, but the atomics we use (`atomicCAS`, `atomicAdd`) are at most 64 bits wide.
2. **Unknown bases**: chr1 contains 18.5 M `N`s in 168 segments; every k-mer that crosses a segment border must be skipped.
3. **Memory**: the table for chr1 needs several GB on a 16 GB card, and copying it back over PCIe is expensive.
4. **Random access**: hashing scatters every update over the whole table, so caches barely help.
5. **Repeats (skew)**: some k-mers (e.g. poly-A, satellites) occur thousands of times $\Rightarrow$ contention on the same slot.

# Datasets (public, FASTA)

## *E. coli* K-12 MG1655

- NCBI RefSeq `NC_000913.3`, complete genome
- Bacterium, the standard lab strain
- 4,641,652 bases, no `N`
- Small: development and correctness tests

## Human GRCh38, chromosome 1

- Ensembl, `homo_sapiens/dna`, chromosome 1
- Largest human chromosome
- 248,956,422 bases, 18,475,410 `N`
- Large: performance evaluation

| | E. coli | GRCh38 chr1 |
|---|---|---|
| A / C / G / T (M) | 1.14 / 1.18 / 1.18 / 1.14 | 67.1 / 48.1 / 48.1 / 67.2 |
| bases after removing `N` | 4,641,652 | 230,481,012 |
| segments (split by `N`) | 1 | 168 |
| 31-mer positions | 4,641,622 | 230,476,809 |
| distinct 31-mers | 4,570,839 | 208,061,649 |

# Version 1: no packing (string keys)

```
for (i = 0; i < dna.length() - k + 1; i++) {
  std::string key = dna.substr(i, k); // copy k chars
  int idx = -1;                       // scan for N
  for (j = 0; j < key.length(); j++)
    if (key[j] == 'N') { idx = j; break; }
  if (idx != -1) { i = idx; continue; }

  uint32_t* p = table.search(key);    // hash 31 bytes
  if (p == nullptr) table.insert(key, 1);
  else              table.insert(key, *p + 1);
}
```

**Cost per k-mer**

- allocate and copy a 31-char string
- scan $k$ chars for `N`
- hash 31 bytes, compare 31 bytes
- key in memory: 32+ bytes

$\Rightarrow O(n \cdot k)$ work and heavy memory traffic.

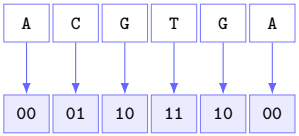
# Version 2: 2-bit packing

**Encoding**: A=00, C=01, G=10, T=11

- 32 bases per `uint64_t`, first base in the top bits
- input is 4× smaller (chr1: 58 MB)
- a 31-mer is one 62-bit integer
- complement = `x ^ 3`

**Unknown bases**

- `N` runs are removed while loading
- `breaks`: positions where a new segment starts




word = 00 01 10 11 10 00 ... (64 bits)

**Rolling key**: next k-mer in $O(1)$

$$\text{key} \leftarrow ((\text{key} \ll 2) \mid \text{base}) \;\&\; (2^{2k} - 1)$$

# Version 2: implementation (best sequential)

```
std::unordered_map<uint64_t, uint64_t> mp;
uint64_t mask = (1ull << (2 * k)) - 1;      // keep the last k bases
uint64_t key = 0; uint32_t run = 0; size_t b = 0;
for (size_t i = 0; i < n; i++) {
  if (b < breaks.size() && breaks[b] == i) { key = 0; run = 0; b++; }   // new segment after N
  size_t bits = (words[i / 32] >> (62 - 2 * (i & 31))) & 3;           // base i from the packed array
  key = ((key << 2) | bits) & mask;                                    // rolling key
  if (++run >= k) mp[key]++;                                           // a full k-mer is ready
}
```

**Why this is the best sequential baseline**

- $O(n)$: one shift/or/and per base
- 8-byte integer keys: cheap hash and compare
- no allocation per k-mer
- single pass, streaming input

**E. coli, single thread** (same compiler flags)

| version | time |
|---|---|
| no pack (string keys) | 2,001 ms |
| 2-bit pack + rolling key | 1,096 ms |

$\Rightarrow$ **1.8× faster**

# Parallel design: one thread per k-mer

1. CPU loads and packs the genome (shared with the CPU version).
2. Copy the packed words and `breaks` to the GPU.
3. Thread $p$ handles the k-mer starting at position $p \in [0, n - k]$:
   - skip it if it crosses an `N` run
   - build its key from 2 words
   - insert-or-increment in a GPU hash table
4. Copy the non-empty slots back.

**Why it maps well to the GPU**

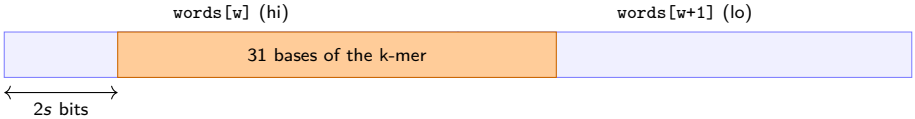
- every position is independent: no rolling dependency
- a warp reads 2–3 consecutive words: **coalesced** input
- the only interaction between threads is the table update $\Rightarrow$ solved with atomics, no locks

**Hash table (written from scratch)**

open addressing, linear probing, static size, load $\leq 0.5$

# Building a key without a rolling dependency

A 31-mer starting at slot $s = p \bmod 32$ of word $w = \lfloor p/32 \rfloor$ spans at most 2 words.




$$x = (\text{hi} \ll 2s) \mid (\text{lo} \gg (64 - 2s)), \qquad \text{key} = x \gg (64 - 2k) \qquad (s = 0 \Rightarrow x = \text{hi})$$

**N check**: the k-mer covers $[p, p + k - 1]$ and is invalid if a segment starts at $b$ with $p < b \leq p + k - 1$. Binary search in the sorted `breaks` array (168 entries for chr1, always in cache).

# Kernel implementation

```
__global__ void gpu_kmer(KeyValue* table, uint64_t cap, uint64_t* words, uint64_t num_words,
                         uint64_t n, uint32_t k, const uint64_t* breaks, uint64_t num_breaks) {
  uint64_t p = (uint64_t)blockIdx.x * blockDim.x + threadIdx.x;
  if (n < k || p > n - k) return;
  uint64_t left = 0, right = num_breaks;                          // first break > p
  while (left < right) { uint64_t mid = (left + right) / 2;
    if (breaks[mid] <= p) left = mid + 1; else right = mid; }
  if (left < num_breaks && breaks[left] < p + k) return;          // crosses an N run

  uint64_t w = p / 32, s = p & 31, hi = words[w], lo = (w + 1 < num_words) ? words[w + 1] : 0;
  uint64_t x = s ? (hi << (2 * s)) | (lo >> (64 - 2 * s)) : hi;
  uint64_t key = x >> (64 - 2 * k);

  uint64_t slot = hash(key, cap);                                 // fmix64(key) & (cap - 1)
  while (true) {
    uint64_t cur = table[slot].key;
    if (cur == key) { atomicAdd(&table[slot].val, 1); return; }
    if (cur == GPU_HASH_EMPTY) {
      uint64_t prev = atomicCAS(&table[slot].key, GPU_HASH_EMPTY, key);
      if (prev == GPU_HASH_EMPTY || prev == key) { atomicAdd(&table[slot].val, 1); return; }
    }
    slot = (slot + 1) & (cap - 1);                                // linear probing
  }
}
```

# Why the concurrent insert is correct

**Two atomics, two jobs**

- `atomicCAS` on the key claims an empty slot
- `atomicAdd` on the count increments it

| CAS returns | meaning |
|---|---|
| `EMPTY` | this thread claimed the slot |
| own key | another thread inserted it first |
| other key | lost the race ⇒ probe on |

**Invariants**

- a key slot changes once: `EMPTY` → k-mer (no deletes)
- all threads with key $X$ follow the same probe sequence ⇒ stop at the same slot ⇒ no duplicates
- key and count need not change together: counts are read after the kernel ⇒ no 128-bit atomic needed
- `EMPTY` = `0xFF..FF` is never a 62-bit key

**Verification**

Test harness: 10 synthetic cases (hot keys, `N` at edges, segments shorter than $k$, table at load 0.9) × repeated runs, plus E. coli and chr1: **every k-mer and count matches the CPU reference**.

# Work–span: why the parallel solution is work-optimal

**Best sequential** (rolling key + hash map)

$$T_{\text{seq}}(n) = O(n)$$

**Parallel work** ($n$ bases, table $\leq 4n$ slots)

- table reset and result compaction: $O(n)$
- per k-mer: N check $O(\log B)$ + key $O(1)$ + insert $O(1)$ expected (load $\leq 0.5$)

$$W(n) = O(n \log B) = O(n)$$

$B$ = number of `N` runs (168): a constant, $\log B \approx 8$.

**Work-optimal**

$W = \Theta(T_{\text{seq}})$: the GPU does the same $O(n)$ work, only a constant more per k-mer. A lost `atomicCAS` just moves to the next slot, so no work is repeated.

**Span**

$$S = O(\log B + L_{\max} + c_{\max})$$

longest probe chain $L_{\max}$ and most repeated k-mer $c_{\max}$ (its atomics serialize): tiny compared with $n \Rightarrow$ massive parallelism $W/S$.

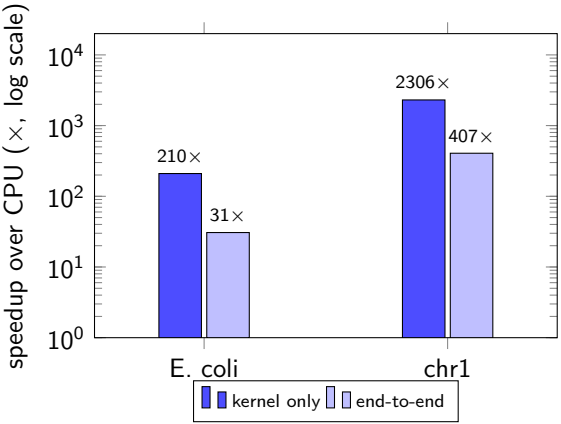
# Benchmark setup

- GPU: NVIDIA GeForce RTX 5060 Ti 16 GB (36 SMs, 32 MB L2), CUDA 13.4
- CPU: Intel Core i7-14700F, 32 GB RAM, Windows 11, MSVC `/O2` (via `nvcc -O3`)
- CPU baseline: packed rolling key + `std::unordered_map`, single thread
- GPU: `gpu_kmer` kernel, table = $2^{\lceil \log_2 2n \rceil}$ slots of 16 B
- Timing
  - GPU: CUDA events, median of 10 runs after 1 warm-up
  - CPU: `std::chrono`, median of 5 runs (E. coli), 1 run (chr1)
  - reading and packing FASTA is shared and **not** timed
  - end-to-end GPU = H2D + table reset + kernel + D2H of the result
- Both results are compared (distinct and total): **MATCH** on both datasets
- chr1 GPU times come from a GPU-only run: right after the 10 GB CPU run the GPU timings were disturbed by memory pressure

# Results: CPU vs GPU

| time (ms) | E. coli | GRCh38 chr1 |
|---|---|---|
| CPU `unordered_map` | 783.9 | 485,613 |
| GPU H2D words + breaks | 0.14 | 4.4 |
| GPU table reset | 0.71 | 22.4 |
| GPU kernel | 3.74 | 210.6 |
| GPU D2H result | 20.7 | 955.0 |
| GPU end-to-end | 25.5 | 1,194.6 |
| throughput CPU (M k-mers/s) | 5.9 | 0.47 |
| throughput GPU kernel (M k-mers/s) | 1,242 | 1,095 |

# Speedup of the GPU




- The kernel is **2–3 orders of magnitude** faster than one CPU core.
- End-to-end is dominated by **copying the result back** (D2H is the largest GPU phase).
- H2D is tiny: the packed input is 4× smaller than text.
- The CPU drops from 5.9 to 0.47 M k-mers/s on chr1: its map grows past 10 GB, so almost every update misses the caches.

# Arithmetic intensity of the kernel

**Per k-mer**

- compute: ∼60 integer ops (N check, key, hash, probe)
- memory: ∼64 B from DRAM (one 32 B sector read + written back for the slot)

$$\text{AI} \approx \frac{60 \text{ ops}}{64 \text{ B}} \approx \mathbf{0.9 \text{ ops/B}}$$

**GPU balance point**

$$\frac{\text{peak ops}}{\text{bandwidth}} \approx \frac{11.8 \text{ T ops/s}}{383 \text{ GB/s}} \approx \mathbf{31 \text{ ops/B}}$$

$\text{AI} \approx 0.9 \ll 31 \Rightarrow$ the kernel waits for memory, not for the ALUs: **memory-bound**.

# Design choices already in the basic kernel

- **2-bit packed input**: 58 MB for chr1, coalesced reads, cheap H2D.
- **Good hash (`fmix64`)**: identity hash + power-of-two table would use only the last $\sim$14 bases $\Rightarrow$ huge clusters.
- **Power-of-two capacity**: `& (cap-1)` instead of a 64-bit modulo.
- **Load factor** $\leq 0.5$: short linear-probing chains.
- **Read before CAS**: existing k-mers cost one load + one `atomicAdd`.
- **Static table, no rehash**: size from the upper bound $n - k + 1$.

# Does the histogram trick "privatization" help?

In the histogram lab each block keeps a private copy in shared memory and merges it once. It pays off only if threads of one block hit the **same** bins.

Measured: distinct 31-mers inside a window of $W$ consecutive positions (= global updates left after privatization)

| window $W$ | E. coli: updates saved | chr1: updates saved |
|---|---|---|
| 32 (warp) | 0.00% | 0.04% |
| 256 (block) | 0.04% | 0.10% |
| 1024 | 0.10% | 0.20% |
| 8192 | 0.13% | 0.93% |

**Conclusion**

Neighbouring 31-mers are almost always different: privatizing the k-mer table saves $\leq 1\%$ of the work but adds shared-memory probing and synchronization.

# Planned optimizations

**End-to-end**

- **Pinned host memory** (`cudaMallocHost`) for the result copy: D2H is the largest phase today

**Scale**

- Whole human genome (3.1 G bases): the table no longer fits in 16 GB $\Rightarrow$ split the k-mers by hash and count them in rounds

**Comparisons**

- NVIDIA cuCollections `static_map`
- multi-core CPU baseline

# Summary

- Exact 31-mer counting needs a **hash table**: a histogram would need 18 EB.
- Best sequential: 2-bit packing + rolling key, $O(n)$ with integer keys.
- GPU: one thread per position, 2-word key extraction, `N` check by binary search, lock-free open addressing with `atomicCAS` + `atomicAdd`.
- Verified k-mer by k-mer against the CPU on E. coli and human chr1.
- Work-optimal ($W = O(n)$) with span $O(\log B + L_{\max} + c_{\max})$.
- Memory-bound: arithmetic intensity $\approx 0.9$ ops/B, far below the GPU balance point ($\approx 31$).
- Kernel speedup **2,306**$\times$ and end-to-end **407**$\times$ on chr1.

# Questions?

Thank you